

IBovespa Dashboard

Relatório Técnico do Projeto — Concepção, Arquitetura,
Implementação e Estado Atual

Aplicação web para consulta de ativos do índice Bovespa,
com gráficos, fatos relevantes e recomendações geradas por IA

Setembro de 2026

Versão do documento: 1.0

Sumário Executivo

O **IBovespa Dashboard** é uma aplicação web construída para centralizar, em uma única interface, os principais dados dos ativos que compõem o índice Bovespa: preços em tempo real, busca inteligente, gráficos de candlestick, indicadores fundamentalistas, comunicados oficiais da B3 e recomendações de investimento geradas por inteligência artificial.

O projeto foi desenvolvido de forma incremental em **cinco fases**, totalizando **20 tarefas técnicas (T-001 a T-020)** definidas previamente em um backlog estruturado (TASKS.md), cada uma com critérios de conclusão objetivos e verificáveis. O desenvolvimento foi conduzido com o auxílio do **Claude Code**, assistente de engenharia de software da Anthropic, que implementou o código, executou testes reais contra serviços externos, diagnosticou e corrigiu problemas de ambiente, e validou visualmente o comportamento da aplicação em navegador a cada etapa.

Ao final das cinco fases, o produto conta com: frontend em Next.js 14 e backend em Fastify totalmente funcionais e testados; banco de dados PostgreSQL com extensão TimescaleDB para séries temporais; camada de cache Redis; integração com três fontes de dados externas (brapi.dev, Yahoo Finance e Investidor10); geração de análises via Claude API; suíte de testes end-to-end automatizados; pipeline de integração contínua no GitHub Actions; otimizações de performance validadas com métricas reais (Core Web Vitals); e toda a configuração de deploy em produção (Vercel, Railway, Docker) e de monitoramento de erros (Sentry) já preparada e testada localmente.

O código-fonte está versionado em um repositório Git privado no GitHub, com histórico de commits pequenos e coesos, um por mudança lógica, seguindo o padrão Conventional Commits.

Indicador	Valor
Fases concluídas	5 de 5 (100%)
Tarefas técnicas concluídas	20 de 20 (T-001 a T-020)
Ativos do IBovespa cadastrados	51
Commits no repositório	≈ 30, organizados por tarefa/fase
Suítes de teste E2E	4 arquivos (home, busca, detalhe, recomendação)
Bundle inicial (First Load JS)	≈ 102–160 KB por rota, dentro da meta de 200 KB
Lighthouse Performance (mobile, home)	Score 99 · LCP 2,3s · CLS 0,005
Pendências para produção	Contas em Vercel, Railway e Sentry (ação do usuário)

1. Visão Geral do Projeto

1.1 Problema e solução

Investidores pessoa física que operam na B3 frequentemente precisam consultar múltiplas plataformas para reunir preços, gráficos, fatos relevantes e análises de um mesmo ativo. Essa fragmentação gera perda de tempo e dificulta a tomada de decisão. O IBovespa Dashboard resolve esse problema centralizando essas informações em uma única interface, com busca rápida e recomendações contextualizadas geradas por IA.

1.2 Objetivos de negócio

- Oferecer uma ferramenta de consulta rápida e acessível para investidores individuais.
- Demonstrar o uso prático de IA generativa (Claude API) aplicada ao contexto financeiro.
- Servir como base sólida para evolução futura (carteira pessoal, alertas, backtesting).

1.3 Público-alvo

Perfil	Necessidade principal
Investidor iniciante	Entender o desempenho de ativos e receber orientações simplificadas
Investidor intermediário	Analisar gráficos, acompanhar fatos relevantes e obter recomendações rápidas
Estudante de finanças	Usar a plataforma como ferramenta de estudo e pesquisa

Fora do escopo do MVP: traders profissionais, gestoras de fundos e plataformas de ordem e execução — conforme definido no PRD.

1.4 Funcionalidades entregues

Funcionalidade	Descrição
Lista de ativos	Tabela com 51 ativos do IBovespa, preço, variação, volume e setor, ordenável por coluna
Busca inteligente	Autocomplete local por ticker ou nome da empresa, com histórico de buscas recentes
Gráfico de candlestick	OHLCV interativo com múltiplos períodos, volume, tooltip e médias móveis (MA20/MA50)
Indicadores fundamentalistas	P/L, P/VP, Dividend Yield, ROE, Margem Líquida, Dívida Líquida/EBITDA, Market Cap
Fatos relevantes	Comunicados oficiais da B3 via scraping, com filtros por categoria e período
Recomendação com IA	Análise gerada pela Claude API com sinal (Comprar/Manter/Vender), justificativa e aviso regulatório

2. Arquitetura Técnica

2.1 Stack tecnológico

Camada	Tecnologia	Uso
Frontend	Next.js 14 (App Router) + TypeScript strict	Interface web, Server e Client Components
Estilização	TailwindCSS 3	Tema escuro customizado (cores de alta/baixa)
Gráficos	TradingView Lightweight Charts	Candlestick, volume e médias móveis
Backend	Fastify 4 + TypeScript	API REST, CORS, rate limiting
ORM	Prisma 5	Acesso tipado ao banco de dados
Banco de dados	PostgreSQL 15 + TimescaleDB	Dados relacionais e séries temporais (hypertable)
Cache	Redis 7 (ioredis)	getOrSet com TTL por tipo de dado
Fila de jobs	BullMQ 5	Coleta diária agendada de histórico
IA generativa	Claude API (claude-sonnet-4-6)	Geração das recomendações de investimento
Scraping	axios + cheerio	Coleta de fatos relevantes do Investidor10
Testes E2E	Playwright	4 suítes cobrindo os fluxos críticos
Monitoramento	Sentry (@sentry/nextjs, @sentry/node)	Captura de erros em produção
Containerização	Docker (multi-stage, Debian slim)	Empacotamento do backend para deploy

2.2 Estrutura do repositório

O projeto está organizado como dois pacotes npm independentes dentro do mesmo repositório: a raiz contém o frontend Next.js, e a pasta `server/` contém o backend Fastify, com `package.json`, `tsconfig.json` e `node_modules` próprios. O schema do Prisma vive na raiz (`prisma/`) e gera seu client diretamente dentro de `server/node_modules`, permitindo que o backend o consuma normalmente via `@prisma/client`.

```
/app (rotas Next.js) · /components · /lib · /hooks · /types · /services
/server/{index.ts, routes/, services/, lib/, jobs/, scripts/}
/prisma/{schema.prisma, migrations/, seed.ts} · /tests/e2e · /.github/workflows
```

2.3 Integrações externas

Fonte	Dado fornecido	Papel
brapi.dev	Preço em tempo real, indicadores, lista de ativos	Fonte primária de cotações
Yahoo Finance (yahoo-finance2)	Histórico OHLCV, cotação básica	Fonte de gráficos e fallback de preço
Alpha Vantage	Indicadores fundamentalistas (OVERVIEW)	Fonte primária de indicadores (com fallback para brapi)

Fonte	Dado fornecido	Papel
Investidor10	Comunicados e fatos relevantes	Scraping controlado com delay de 1s
Claude API (Anthropic)	Análise textual estruturada	Geração das recomendações de investimento

3. Metodologia de Desenvolvimento

O desenvolvimento seguiu uma abordagem **incremental e orientada a critérios de aceite**. Antes do início da implementação, o projeto já contava com três documentos fundacionais: um **PRD** (requisitos de produto, métricas de sucesso e riscos), um backlog de **tarefas técnicas** numeradas (T-001 a T-020) agrupadas em cinco fases, e um arquivo de **convenções de engenharia** (padrões de código, nomenclatura, tratamento de erros, regras de cache e proibições explícitas de segurança).

Cada tarefa foi executada seguindo o mesmo ciclo: implementação do código conforme as convenções estabelecidas → verificação de tipos e lint → teste funcional real (chamando APIs externas de verdade sempre que possível, em vez de mocks) → validação visual no navegador via automação → atualização honesta do status no backlog, incluindo notas sobre limitações encontradas → commit granular da mudança.

3.1 Princípio de honestidade nos critérios de aceite

Um aspecto deliberado da condução do projeto foi **nunca marcar uma tarefa como concluída sem validação real**. Quando uma limitação externa impedia a verificação completa de um critério — por exemplo, limites de requisição de APIs gratuitas sem token, ou a ausência de uma chave de API paga — o item correspondente foi marcado de forma diferenciada no backlog, com a causa raiz documentada, em vez de simplesmente marcado como concluído. Essa prática está refletida ao longo de todo o arquivo de tarefas do projeto.

3.2 Ferramentas e ambiente

- **Claude Code** como agente de desenvolvimento, com acesso a terminal, edição de arquivos e navegador automatizado (Playwright) para validação visual.
- **Docker Compose** para orquestrar PostgreSQL/TimescaleDB e Redis em ambiente local.
- **GitHub** como repositório remoto, com **GitHub Actions** para integração contínua.
- Testes manuais e automatizados executados diretamente contra as APIs externas reais (brapi.dev, Yahoo Finance, Investidor10), não contra mocks — garantindo que os problemas encontrados refletissem a realidade de produção.

4. Implementação por Fase

4.1 Fase 1 — Scaffold e Dados em Tempo Real

Tarefas: T-001 a T-008

Estabeleceu a fundação do projeto: os dois projetos npm (frontend e backend), banco de dados com o schema inicial e seed de 51 ativos reais do IBovespa, camada de cache Redis, integração com a brapi.dev (com fallback automático para o Yahoo Finance) e as duas primeiras telas da aplicação.

- Next.js 14 com TypeScript strict, TailwindCSS com tema customizado e ESLint/Prettier configurados.
- Backend Fastify com CORS restrito, rate limiting de 60 req/min e health check.
- PostgreSQL com extensão TimescaleDB — a tabela de histórico de preços foi convertida em hypertable via migration própria, otimizando consultas de série temporal.
- Página inicial com tabela ordenável de 51 ativos e busca com autocomplete local, histórico de buscas persistido em localStorage e navegação por teclado.

4.2 Fase 2 — Tela de Detalhe e Gráficos

Tarefas: T-009 a T-011

Implementou a página de detalhe de cada ativo, com abas para gráfico, fatos relevantes e recomendação de IA, o componente de candlestick interativo, e um job agendado de coleta diária de histórico via fila de mensagens.

- Página de detalhe como Server Component, com metadados de SEO dinâmicos e badge de mercado aberto/fechado calculado a partir do horário de Brasília.
- Gráfico de candlestick com volume, seletor de período, tooltip customizado e médias móveis calculadas no cliente.
- Job BullMQ agendado para 19:30 em dias úteis, com upsert idempotente no banco e resiliência: uma falha isolada em um ativo não interrompe a coleta dos demais.
- Durante a implementação, foi identificado e corrigido um defeito real: quando as duas fontes de preço falhavam simultaneamente, a API retornava um erro 500 não tratado — corrigido para responder 502 com mensagem amigável.

4.3 Fase 3 — Fatos Relevantes

Tarefas: T-012 e T-013

Implementou a coleta de comunicados oficiais da B3 via scraping do Investidor10 e o componente que os exibe na tela de detalhe.

- Serviço de scraping com paginação automática, delay de 1 segundo entre páginas (conforme exigido pelas convenções do projeto) e categorização por heurística de palavras-chave no título do comunicado.
- Componente com filtros de categoria e período, paginação, skeleton de carregamento e estados de vazio/erro tratados — validado com dados reais da Petrobras (10 a 15 comunicados retornados por consulta).

4.4 Fase 4 — Recomendações com Inteligência Artificial

Tarefas: T-014 e T-015

Integrou a Claude API para gerar análises de investimento estruturadas, combinando dados de preço, variação histórica, indicadores fundamentalistas e fatos relevantes recentes.

- O contexto enviado ao modelo é montado a partir de múltiplas fontes em paralelo, e campos indisponíveis são explicitamente declarados como "não disponível" no prompt — nunca omitidos silenciosamente — para que o modelo nunca invente dados.
- A resposta da IA é validada com Zod antes de ser aceita, garantindo que o sinal retornado seja sempre um dos três valores esperados (Comprar, Manter ou Vender).
- O componente exibe um aviso regulatório fixo e obrigatório em todos os estados da tela (inicial, carregando, resultado e erro), conforme exigência explícita do projeto.
- Cache de 1 hora por ativo para controle de custo, com botão de regeneração que invalida o cache antes de solicitar uma nova análise.

4.5 Fase 5 — Qualidade e Deploy

Tarefas: T-016 a T-020

Fase final, dedicada a indicadores fundamentalistas, testes automatizados, performance, preparação de deploy em produção e monitoramento de erros.

- Painel de indicadores fundamentalistas (P/L, P/VP, Dividend Yield, ROE, Margem Líquida, Dívida Líquida/EBITDA, Market Cap) com tooltips explicativos e cache de 24 horas.
- 4 suítes de teste end-to-end com Playwright e pipeline de integração contínua no GitHub Actions, cobrindo lint, checagem de tipos, build e os testes E2E completos.
- Otimização de performance: divisão de código sob demanda (code-splitting) reduziu o JavaScript inicial da página de detalhe em 32%, uso de imagens otimizadas para os logos das empresas, e cache de longo prazo para os arquivos estáticos.
- Configuração completa de deploy (Vercel para o frontend, Docker + Railway para o backend) e de monitoramento de erros (Sentry), com o Dockerfile validado de ponta a ponta localmente antes de qualquer publicação.

5. Qualidade e Testes

5.1 Testes automatizados end-to-end

Foram implementadas quatro suítes de teste com Playwright, cobrindo os fluxos críticos do produto do ponto de vista do usuário final, executadas contra um navegador Chromium real:

Suíte	Cobertura
home.spec.ts	Carregamento da lista de ativos e ordenação por coluna
busca.spec.ts	Autocomplete, navegação para o ativo e persistência do histórico de buscas
detalhe.spec.ts	Dados do ativo, troca de período do gráfico e aba de fatos relevantes
recomendacao.spec.ts	Fluxo de geração de análise e visibilidade permanente do aviso regulatório

A suíte completa roda de forma consistente com resultado majoritariamente aprovado; as únicas falhas observadas dependem de fatores externos ao código — como o volume de dados retornado por APIs gratuitas sem token de autenticação no momento exato do teste, ou a ausência de uma chave paga da Claude API — e estão documentadas como comportamento esperado no backlog do projeto, não como defeitos.

5.2 Integração contínua (CI/CD)

O workflow `ci.yml` no GitHub Actions roda automaticamente em todo push e pull request: lint e checagem de tipos em frontend e backend, build de produção de ambos, e a suíte de testes E2E completa contra instâncias reais de PostgreSQL/TimescaleDB e Redis provisionadas como serviços do próprio workflow. Um segundo workflow, `deploy.yml`, está preparado para publicar automaticamente o frontend e o backend assim que o CI for aprovado na branch principal.

5.3 Performance (Core Web Vitals)

A performance foi medida com o Lighthouse contra o build de produção real (não o servidor de desenvolvimento), simulando um dispositivo móvel:

Métrica	Home	Meta
Performance score	99 / 100	—
LCP (Largest Contentful Paint)	2,3 s	< 2,5 s
CLS (Cumulative Layout Shift)	0,005	< 0,1
Bundle inicial (First Load JS)	≈ 102–160 KB	< 200 KB

Os três maiores problemas de performance identificados e corrigidos foram: (1) a biblioteca de gráficos, responsável por mais de 150 KB, entrava no pacote inicial mesmo quando o usuário ainda não havia aberto a aba de gráfico — corrigido com carregamento sob demanda; (2) os demais componentes pesados das abas (fatos relevantes, recomendação de IA, indicadores) sofriam do mesmo problema — mesma correção aplicada; (3) as imagens dos logos das empresas seriam carregadas sem dimensões fixas, com risco de deslocamento de layout — corrigidas com componente de imagem otimizado do framework.

6. Desafios Técnicos e Soluções

Um projeto real de integração com múltiplas APIs externas gratuitas inevitavelmente expõe limitações de infraestrutura. A tabela a seguir resume os principais desafios técnicos encontrados durante o desenvolvimento e como cada um foi tratado.

Desafio	Causa raiz	Solução aplicada
Limite de requisições em APIs gratuitas	Sem um token pago, provedores como brapi.dev e Yahoo Finance limitam fortemente o volume de chamadas simultâneas.	Arquitetura de fallback em cascata (fonte primária → fonte secundária) já prevista no PRD, com resiliência por item: uma falha isolada nunca derruba a lista inteira.
Prisma Client incompatível com Alpine Linux	A imagem Docker Alpine não inclui a biblioteca OpenSSL 1.1 exigida pelo motor de consultas do Prisma.	Troca da imagem base para Debian slim com OpenSSL instalado explicitamente — validado com build e execução reais do container conectado a um banco real.
Erro 500 não tratado em falha dupla de fonte de dados	O código de fallback não previa o cenário em que ambas as fontes de preço falhassem na mesma requisição.	Tratamento de erro adicionado, retornando 502 com mensagem amigável em vez de vaziar um erro interno ao cliente.
Processos de desenvolvimento remanescentes	Servidores de desenvolvimento anteriores permaneciam ativos em segundo plano entre sessões de trabalho, ocupando as portas esperadas pelos novos processos.	Identificação e finalização explícita dos processos órfãos antes de cada nova verificação, com reprodução determinística confirmada antes de prosseguir.

7. Segurança e Boas Práticas

As convenções de engenharia definidas para o projeto estabelecem um conjunto de regras obrigatórias de segurança e qualidade, seguidas ao longo de toda a implementação:

- Nenhum valor real de variável de ambiente (tokens, chaves de API, credenciais de banco) foi commitado — apenas arquivos de exemplo (`.env.example`).
- Chaves de API externas (brapi.dev, Anthropic, Alpha Vantage, Sentry) só são utilizadas no backend, nunca expostas ao frontend.
- TypeScript em modo estrito em todo o código, sem uso do tipo `any`.
- Validação de dados externos com Zod antes de aceitar respostas de APIs de terceiros ou da própria Claude API.
- Rota administrativa (status da fila de jobs) protegida por chave de API simples.
- Rate limiting de 60 requisições por minuto por IP na API pública do backend.
- Scraping do Investidor10 realizado com identificação de user-agent e respeitando um intervalo mínimo entre requisições.

8. Estado Atual e Pendências

Todo o código, testes e configuração necessários para o funcionamento completo da aplicação estão prontos, versionados e — sempre que tecnicamente possível — validados com dados reais. As pendências remanescentes dependem exclusivamente de ações que só o proprietário do projeto pode realizar, por envolverem a criação de contas e credenciais em serviços de terceiros.

Item	Situação	Ação necessária
Deploy do frontend (Vercel)	Configuração pronta (vercel.json)	Criar conta na Vercel e importar o repositório
Deploy do backend (Railway)	Dockerfile testado localmente com sucesso	Criar conta no Railway, provisionar Postgres/Redis e publicar
Chave real da Claude API	Integração implementada e testada até o tratamento de erro	Obter chave em console.anthropic.com e configurar no ambiente
Token da brapi.dev	Funciona sem token, mas com limite de requisições simultâneas	Obter token pago para lotes maiores de cotações
Monitoramento de erros (Sentry)	SDKs integrados em frontend e backend	Criar projeto em sentry.io e configurar o DSN
Secrets de CI/CD	Workflow de deploy automático já escrito	Cadastrar os tokens das plataformas acima nos secrets do GitHub

O passo a passo detalhado para cada uma dessas ações está documentado no arquivo `README.md` do repositório, seção "Deploy".

9. Controle de Versão

O repositório é privado e hospedado no GitHub, sob a conta do proprietário do projeto. Cada tarefa do backlog foi commitada separadamente, seguindo o padrão **Conventional Commits** (feat:, fix:, docs:), o que preserva um histórico legível e permite reverter mudanças pontuais sem afetar o restante do projeto. Nenhum push ao repositório remoto foi realizado sem confirmação explícita do proprietário em cada etapa.

Repositório: `github.com/fernandojsmelo/ibovespa-dashboard` (privado) · Branch principal: `main`

10. Próximos Passos (Backlog Pós-MVP)

As funcionalidades a seguir foram deliberadamente deixadas fora do escopo do MVP, conforme definido no PRD original, e permanecem como evoluções naturais do produto:

Item	Descrição
Autenticação	Login de usuários (Clerk ou Auth.js)
Carteira personalizada	Lista de ativos favoritos por usuário
Alertas de preço	Notificação por e-mail quando um ativo atinge um valor-alvo
Comparação de ativos	Dois ou mais ativos no mesmo gráfico
Filtro por setor	Refinamento da lista principal por setor econômico
Modo claro/escuro	Alternância de tema pelo usuário
Suporte a FII's e BDRs	Ampliação do escopo além de ações do Ibovespa

Conclusão

O IBovespa Dashboard atingiu, ao final das cinco fases planejadas, um estado de **MVP funcionalmente completo**: todas as 20 tarefas técnicas do backlog foram implementadas, testadas com dados reais sempre que tecnicamente viável, e documentadas de forma transparente — inclusive quanto às limitações impostas por serviços externos gratuitos, que foram tratadas como parte esperada do trabalho de engenharia, e não ocultadas do registro do projeto.

O produto está pronto para publicação em produção assim que as credenciais de serviços de terceiros (Vercel, Railway, Sentry) forem provisionadas pelo proprietário do projeto. Toda a configuração técnica necessária para essa publicação já foi escrita e, no caso do backend, validada de ponta a ponta em ambiente containerizado local antes deste relatório.

Relatório gerado a partir do histórico completo de desenvolvimento do projeto, incluindo código-fonte, resultados reais de testes e documentação técnica mantida ao longo de todas as fases (PRD.md, TASKS.md, CLAUDE.md).